`interval`. By default, the cluster pings each `ProjectionBehavior` every 10 seconds to make sure it is running. When rebalancing occurs, those behaviors are stopped. With this keep-alive interval, you make sure those behaviors are started afterward.

`ProjectionBehavior.Stop` is the message sent to the actor when the behavior is rebalanced or the cluster shuts down. This can be useful if you want to perform actions before a projection stops, like closing the connection to the database.

10.3.6 All the main parts

To complete the application, you need to create three things in the `main` method:

- The `ActorSystem`
- The projection
- The database connection

These are shown in listing 10.12. By now, you are probably familiar with instantiating the `ActorSystem`. Here you create it with a default configuration and add a `port` variable so you can run multiple nodes locally if necessary to avoid port conflicts. The only thing we haven't covered is the database connection—the `ScalikeJdbcSetup.init(system)`—but that has little to do with Akka, and explaining it here would complicate the discussion. The code is available in the Git repository if you want to take a closer look.

Listing 10.12 Complete entry point of the application

```
import scala.util.control.NonFatal
import org.slf4j.LoggerFactory
import akka.actor.typed.ActorSystem
import example.repository.scalike.ScalikeJdbcSetup

object Main {

  val logger = LoggerFactory.getLogger(Main + "")

  def main(args: Array[String]): Unit = {
    logger.info("initializing system")
    val system = if (args.isEmpty) {
      initActorSystem(0)                                ❶
    } else {
      initActorSystem(args(0).toInt)                    ❷
    }

    try {
      ScalikeJdbcSetup.init(system)                     ❸
      initProjection(system)                            ❹
    } catch {
      case NonFatal(ex) =>    #E                        ❺
        logger.error(s"terminating by NonFatal Exception", ex)
        system.terminate()
    }
  }

  def initActorSystem(port: Int):ActorSystem[_] = {     ❻
    val config = ConfigFactory
      .parseString(s"""
      akka.remote.artery.canonical.port=$port
      """)
      .withFallback(ConfigFactory.load())
```